

Statistical Inference for RNA-seq Data Analysis

Yongjin Park

22 January 2026

Announcement

Check out your teams!

ASDA #1: Blood RNAs for Febrile Illness Diagnosis

- Scoping review identified 286 RNA signatures from literature
- Signatures diagnose or predict outcomes of febrile illnesses (sepsis, TB, infections)
- Goal: Characterize and understand potential sources of biases
- Meta-analysis across many different studies

ASDA #2: Breath print for Lung Cancer Detection

- Breath samples from 500+ participants (lung cancer vs. healthy controls)
- Volatile organic compounds (VOCs) analyzed via mass spectrometry
- High-dimensional data: ~500-1500 compounds per sample
- Goal: Build analysis pipeline for biomarker discovery and classification

Learning objectives of RNA-seq analysis

How do we get sequencing data?

- High-throughput sequencing data
 - data generation scheme
 - quantification
- Data normalization
 - intrinsic biases
 - gene-level adjustment
 - gene and sample adjustment
- Over-dispersed nature
 - count data
 - transformation

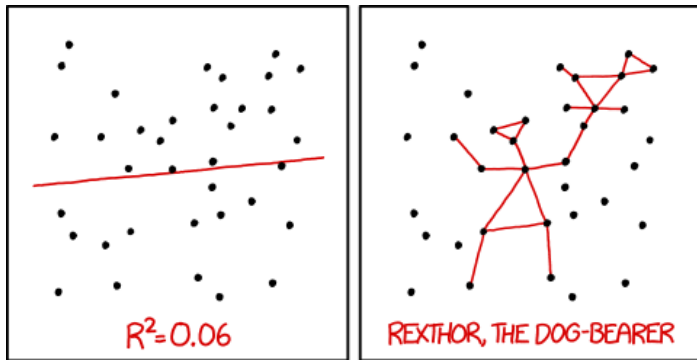
How can we fit a model to sequencing data?

- Linear modelling
 - a Gaussian linear model
 - a generalized linear model
- Multivariate linear models
 - a column space view
- Model-based inference
 - maximum likelihood learning
 - parametric inference
- Revisit RNA-seq
 - [DESeq2](#) vs. [limma-voom](#)
 - modelling and estimation

Today's lecture

- 1 What is a linear regression model?
- 2 Statistical inference for linear regression
- 3 Generalized linear models for RNA-seq
- 4 The limma-voom approach

What is a linear regression?

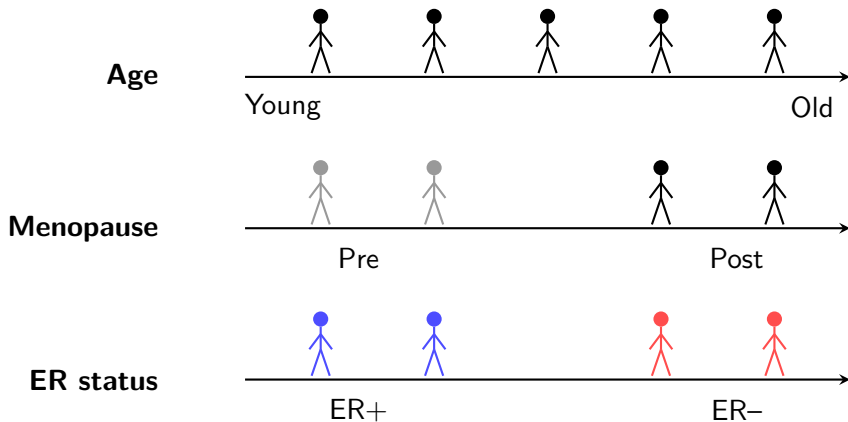


I DON'T TRUST LINEAR REGRESSIONS WHEN IT'S HARDER
TO GUESS THE DIRECTION OF THE CORRELATION FROM THE
SCATTER PLOT THAN TO FIND NEW CONSTELLATIONS ON IT.

A quick survey on your peers' experience

- ① How many people used a linear regression?
- ② Why is it called “regression?”
- ③ How did you know that your regression worked or not?

TCGA BRCA example: three axes of variation



Key question: multiple sources of variation

- Can we identify multiple independent disease mechanisms?
- How do they manifest in observed transcriptional profiles?

- ① **ER status:** Tumor-intrinsic differences (ER+ vs ER-)
- ② **Age:** age effects in general population
- ③ **Menopause:** pre- vs. post-menopause

They are all correlated... but not necessarily the same

- Post-menopausal but young (e.g., surgical menopause)
- Pre-menopausal but older (late menopause)

From 1ect03: Describe your model by stitching together random variables

Represent joint probability distributions using graphs

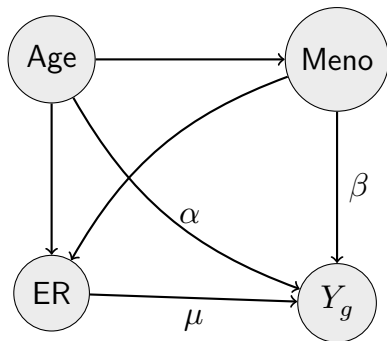
Nodes = Random variables

- Observed (shaded)
- Latent/hidden (open)
- Parameters (dots or small nodes)

Edges = Dependencies

- Arrow $\rightarrow$ = causal/generative direction
- No arrow = correlation/association

Overall graphical model



Observed variables:

- Age (continuous), Menopause (categorical), ER status (categorical)
- Gene expression matrix Y_{ig} for a gene g in a sample i .

Parameters (open):

- μ : ER status effect
- α : direct age effect
- β : menopause effect

TCGA BRCA data from recount3

We will use The Cancer Genome Atlas (TCGA) breast cancer data

```
data.dir <- "../data/recount3"
dir.create(data.dir, recursive = TRUE, showWarnings = FALSE)
.file <- file.path(data.dir, "tcga_brca_rse.rds")

if.needed(.file, {
  tcga.proj <- subset(available_projects(),
                     file_source == "tcga")
  brca.info <- tcga.proj[tcga.proj$project == "BRCA", ]
  rse.brca <- create_rse(brca.info)
  saveRDS(rse.brca, .file)
})
rse <- readRDS(.file)
```

After alignment: SummarizedExperiment

		samples (columns)						
		+-----+						
			S1	S2	S3	...		
		+-----+	+-----+					
genes (rows)	G1		10	25	18	...		<- assays (counts)
	G2		5	12	8	...		
	...		...	...	...	...		
		+-----+	+-----+					

rowData: genomic coordinates (chr, start, end, strand)

colData: sample metadata (tumor/normal, clinical info)

Here is gene expression count

```
raw.counts <- assay(rse, "raw_counts")
raw.counts[1:3, 1:3]
```

```
##                                0b1cf6de-f98f-4020-baa7-c4fc518e5f00
## ENSG00000278704.1                                0
## ENSG00000277400.1                                0
## ENSG00000274847.1                                0
##                                20c90e97-e900-488e-af6a-6648d9a8bb01
## ENSG00000278704.1                                0
## ENSG00000277400.1                                0
## ENSG00000274847.1                                0
##                                0a2fa7c7-0810-4d65-b16c-4a1792714802
## ENSG00000278704.1                                0
## ENSG00000277400.1                                0
## ENSG00000274847.1                                0
```

Gene/row annotations

```
rowData(rse) %>% head(3)
```

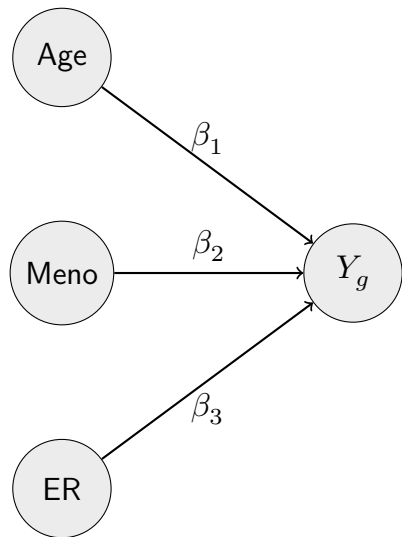
```
## DataFrame with 3 rows and 10 columns
##           source      type bp_length      phase      gene_id
##           <factor> <factor> <numeric> <integer>    <character>
## ENSG00000278704.1 ENSEMBL   gene      2237        NA ENSG00000278704.1
## ENSG00000277400.1 ENSEMBL   gene      2179        NA ENSG00000277400.1
## ENSG00000274847.1 ENSEMBL   gene      1599        NA ENSG00000274847.1
##           gene_type  gene_name      level havana_gene
##           <character> <character> <character> <character>
## ENSG00000278704.1 protein_coding BX004987.1      3      NA
## ENSG00000277400.1 protein_coding AC145212.2      3      NA
## ENSG00000274847.1 protein_coding AC145212.1      3      NA
##           tag
##           <character>
## ENSG00000278704.1      NA
## ENSG00000277400.1      NA
## ENSG00000274847.1      NA
```

Sample/column annotations

```
colData(rse) %>% as.data.frame() %>% select(1:3) %>% head(3)
```

```
##                                rail_id
## 0b1cf6de-f98f-4020-baa7-c4fc518e5f00 106628
## 20c90e97-e900-488e-af6a-6648d9a8bb01 106636
## 0a2fa7c7-0810-4d65-b16c-4a1792714802 106645
##
##                                external_id study
## 0b1cf6de-f98f-4020-baa7-c4fc518e5f00 0b1cf6de-f98f-4020-baa7-c4fc518e5f00 BRCA
## 20c90e97-e900-488e-af6a-6648d9a8bb01 20c90e97-e900-488e-af6a-6648d9a8bb01 BRCA
## 0a2fa7c7-0810-4d65-b16c-4a1792714802 0a2fa7c7-0810-4d65-b16c-4a1792714802 BRCA
```

A probabilistic model on gene expression Y



Linear model for gene g :

$$Y_{ig} = \beta_{1g} \cdot X_{i1} + \beta_{2g} \cdot X_{i2} + \beta_{3g} \cdot X_{i3} + \epsilon_{ig}$$

where $\epsilon_{ig} \sim N(0, \sigma_g^2)$

- X_{i1} : Age for individual i
- X_{i2} : Menopause status for individual i
- X_{i3} : ER status for individual i
- $\beta_{1g}, \beta_{2g}, \beta_{3g}$: Coefficients for gene g

Encoding categorical variables

Categorical variables need to be converted to numbers for the linear model

Menopause status (X_{i2}):

- Pre-menopause: $X_{i2} = 0$
- Post-menopause: $X_{i2} = 1$

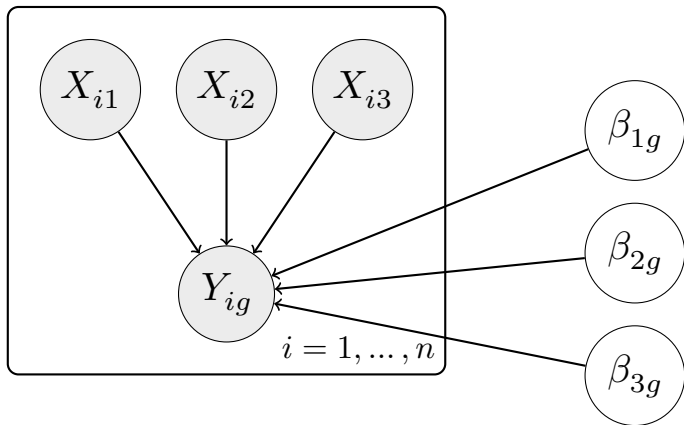
ER status (X_{i3}):

- ER+: $X_{i3} = 0$
- ER-: $X_{i3} = 1$

Interpretation: The coefficient represents the **difference** in expression between the two groups

- β_{2g} : Post-menopause vs. Pre-menopause difference for gene g
- β_{3g} : ER- vs. ER+ difference for gene g

More explicit about the parameters



We can write it down as a linear equation

If it is deterministic and linear:

$$Y_{ig} = X_{i1}\beta_{1g} + X_{i2}\beta_{2g} + X_{i3}\beta_{3g}$$

- Are we treating Y as a random variable?
- If not, what should we do?

Adding uncertainty to the linear equations

If it is stochastic and linear:

$$Y_{ig} = X_{i1}\beta_{1g} + X_{i2}\beta_{2g} + X_{i3}\beta_{3g} + \epsilon_{ig}$$

- What could be the distribution of ϵ ?
- Can we have a single ϵ , namely $\epsilon_{ig} = \epsilon^*$ for all i and g ?

Building the response vector $\mathbf{y}_g$

For a single gene g , the response vector contains expression values across all n samples:

$$\mathbf{y}_g = \begin{pmatrix} Y_{g1} \\ Y_{g2} \\ \vdots \\ Y_{gn} \end{pmatrix}$$

Example: For a gene in our TCGA BRCA dataset:

```
rowData(rse)[17709, 1:5]      # gene name etc.
```

```
## DataFrame with 1 row and 5 columns
```

```
##           source      type bp_length      phase      gene_id
##      <factor> <factor> <numeric> <integer>    <character>
## ENSG00000065361.14  HAVANA      gene      8658          NA ENSG00000065361.14
```

```
yy <- raw.counts[17709, ] # Extract some gene
head(yy, 10)              # Show first 10 values
```

```
## 0b1cf6de-f98f-4020-baa7-c4fc518e5f00 20c90e97-e900-488e-af6a-6648d9a8bb01
##                                     3099081                                     2603039
## 0a2fa7c7-0810-4d65-b16c-4a1792714802 cb2498a3-ade5-44a1-9df8-c611cde2ae03
##                                     1832208                                     2384985
## 30f32969-5278-4066-bb82-7400cdce8807 f9e194a1-e5c1-4a1a-840b-4e0246640d09
##                                     578793                                     2921777
## 2a9ed6f1-3f93-43cf-9e47-6690781e1009 8f349f98-4d93-4f97-ac6b-50d2bf072b0c
##                                     1360049                                     2591677
## 46c0f948-2eac-41f7-b490-fbca1724d710 6203b63e-6d3f-4399-b524-9573a9bd1e11
##                                     4999830                                     2239177
```

Each entry Y_{ig} represents the expression (read count) of gene g in sample i

Building the design matrix $\mathbf{X}$

The design matrix encodes our covariates (predictors) for all n samples:

$$\mathbf{X} = \begin{pmatrix} 1 & X_{11} & X_{12} & X_{13} \\ 1 & X_{21} & X_{22} & X_{23} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & X_{n1} & X_{n2} & X_{n3} \end{pmatrix} = \begin{pmatrix} 1 & \text{Age}_1 & \text{Meno}_1 & \text{ER}_1 \\ 1 & \text{Age}_2 & \text{Meno}_2 & \text{ER}_2 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & \text{Age}_n & \text{Meno}_n & \text{ER}_n \end{pmatrix}$$

What is the first column?

TCGA metadata has very long factor names

```
## Menopause status levels:
```

```
## [1] "Pre (<6 months since LMP AND no prior bilateral ovariectomy AND not on estrogen replacement)"
```

```
## [2] "Post (prior bilateral ovariectomy OR >12 mo since LMP with no prior hysterectomy)"
```

```
## [3] "Peri (6-12 months since last menstrual period)"
```

Original names are too verbose for analysis!

We need to shorten them for readability and modeling

Factor name abbreviations we'll use

Variable	Original	Shortened
Menopause	"Pre (<6 months since LMP...)"	"Pre"
	"Post (prior Bilateral...)"	"Post"
	"Peri (6-12 months...)"	"Peri"
ER status	"Positive"	"Positive"
	"Negative"	"Negative"
Sample type	"Primary Tumor"	"Tumor"
	"Solid Tissue Normal"	"Normal"

Example: Building X from TCGA metadata

```
clinical.data <-  
  colData(rse) %>%  
  as.data.frame() %>%  
  select(  
    sample.id = tcga.tcga_barcode,  
    sample.type = tcga.cgc_sample_sample_type,  
    age = tcga.gdc_cases.diagnoses.age_at_diagnosis,  
    menopause = tcga.xml_menopause_status,  
    tumor.stage = tcga.gdc_cases.diagnoses.tumor_stage,  
    ER.status = tcga.xml_breast_carcinoma_estrogen_receptor_status,  
    PR.status = tcga.xml_breast_carcinoma_progesterone_receptor_status,  
    HER2.status = tcga.xml_lab_proc_her2_neu_immunohistochemistry_receptor_status  
  ) %>%  
  mutate(  
    # Shorten menopause status  
    menopause = case when(  

```

Header lines of sample/clinical data

```
head(clinical.data, 5)
```

```
##              sample.id sample.type  age menopause tumor.stage
##              <char>      <char> <int>    <char>    <char>
## 1: TCGA-E9-A249-01A-11R-A169-07      Tumor 16689      Pre    stage iia
## 2: TCGA-BH-A1EX-01A-11R-A13Q-07      Tumor 24544      Post   stage iib
## 3: TCGA-AO-A12A-01A-21R-A115-07      Tumor 17420      Pre    stage iia
## 4: TCGA-AC-A3OD-01B-06R-A220-07      Tumor 25186      Post   stage iib
## 5: TCGA-AC-A23G-01A-11R-A213-07      Tumor 27934      Post    stage iia
##      ER.status PR.status HER2.status
##      <char>    <char>    <char>
## 1:      <NA>    <NA>    <NA>
## 2: Positive Positive Positive
## 3: Positive Positive Negative
## 4: Positive Positive Negative
## 5: Positive Positive Positive
```

A handy function to build a design matrix

```
X <- model.matrix(~ age + menopause + ER.status,  
                  data = clinical.data)  
head(X, 3)
```

```
##      (Intercept)    age menopausePost menopausePre ER.statusNegative  
## 2              1 24544              1              0              0  
## 3              1 17420              0              1              0  
## 4              1 25186              1              0              0  
##      ER.statusPositive  
## 2              1  
## 3              1  
## 4              1
```

Let's write down this linear model in a matrix form

$$\mathbf{y}_g = \mathbf{X}\beta_g + \epsilon_g$$

where:

$$\bullet \mathbf{y}_g = \begin{pmatrix} Y_{g1} \\ Y_{g2} \\ \vdots \\ Y_{gn} \end{pmatrix} \text{ expression values}$$

$$\bullet \beta_g = \begin{pmatrix} \beta_{0g} \\ \beta_{1g} \\ \beta_{2g} \\ \beta_{3g} \end{pmatrix} \text{ coefficients}$$

$$\bullet \mathbf{X} = \begin{pmatrix} 1 & X_{11} & X_{12} & X_{13} \\ 1 & X_{21} & X_{22} & X_{23} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & X_{n1} & X_{n2} & X_{n3} \end{pmatrix}$$

$$\bullet \epsilon_g = \begin{pmatrix} \epsilon_{1g} \\ \epsilon_{2g} \\ \vdots \\ \epsilon_{ng} \end{pmatrix} \text{ errors}$$

Recap of lect02 matrix algebra...

Why is it useful to write them down in matrix?

A design matrix = a column space view

We can rewrite the linear model as:

$$\mathbf{y}_g = \beta_{0g}\mathbf{x}_0 + \beta_{1g}\mathbf{x}_1 + \beta_{2g}\mathbf{x}_2 + \beta_{3g}\mathbf{x}_3 + \epsilon_g$$

where $\mathbf{x}_j$ is the j -th column of $\mathbf{X}$

A design matrix = a column space view

We can rewrite the linear model as:

$$\mathbf{y}_g = \beta_{0g}\mathbf{x}_0 + \beta_{1g}\mathbf{x}_1 + \beta_{2g}\mathbf{x}_2 + \beta_{3g}\mathbf{x}_3 + \epsilon_g$$

where $\mathbf{x}_j$ is the j -th column of $\mathbf{X}$

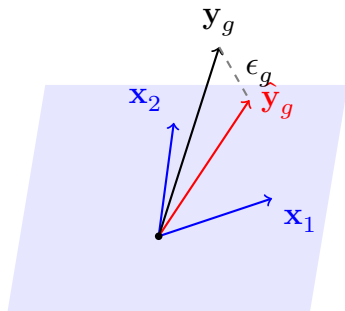
What is a linear regression?

Finding the "best" **linear combination** of **column vectors** to approximate $\mathbf{y}_g$

* Each column $\mathbf{x}_j$ represents a covariate pattern across all samples * The coefficients β_{jg} tell us how much each pattern contributes * The fitted values $\hat{\mathbf{y}}_g = \mathbf{X}\hat{\boldsymbol{\beta}}_g$ lie in the **column space** of $\mathbf{X}$

Side: geometric intuition of column space (lect09)

If the whole prediction space can be made up by a linear combination of two n vectors $\mathbf{x}_1$ and $\mathbf{x}_2$...



Column space of $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2]$

$\hat{\mathbf{y}}_g$ is the **projection** of $\mathbf{y}_g$ onto the column space of $\mathbf{X}$

What have we missed?

$$y_g = \mathbf{X}\beta_g + \epsilon_g$$

What have we missed?

$$y_g = \mathbf{X}\beta_g + \epsilon_g$$

We can assume/model:

$$\epsilon_{ig} \sim \mathcal{N}(0, \sigma^2)$$

How can we estimate the coefficients?

Use standard linear regression (`lm`) to estimate β_g :

```
y_gene <- log1p(raw.counts[17709, ]) # Take expression from gene 17709

fit <- lm(y_gene ~ age + menopause + ER.status, # Fit a linear model
         data = clinical.data)

summary(fit)$coefficients # View coefficients
```

##	Estimate	Std. Error	t value	Pr(> t)
## (Intercept)	1.439550e+01	9.969523e-01	14.43950275	2.981063e-43
## age	9.782245e-06	8.659758e-06	1.12962104	2.588934e-01
## menopausePost	-4.048353e-02	1.605811e-01	-0.25210644	8.010080e-01
## menopausePre	1.540438e-01	1.620356e-01	0.95067898	3.419857e-01
## ER.statusNegative	-9.057548e-01	9.649957e-01	-0.93861016	3.481464e-01
## ER.statusPositive	-6.701076e-02	9.631295e-01	-0.06957607	9.445443e-01

A better way to estimate: use limma



```
library(limma)
```

limma can estimate the parameters across all the genes

```
complete_samples <- complete.cases(clinical.data)
Y_filtered <- log1p(raw.counts[, complete_samples])
design <- model.matrix(~ age + menopause + ER.status,
                      data = clinical.data[complete_samples, ])

fit_limma <- lmFit(Y_filtered, design) # Fit linear models for all genes

fit_limma$coefficients[17709, ] # View coefficients for gene 17709
```

##	(Intercept)	age	menopausePost	menopausePre
##	1.346999e+01	9.945817e-06	-3.169716e-02	1.831340e-01
##	ER.statusPositive			
##	8.391555e-01			

Getting the top genes with topTable

After fitting, use empirical Bayes moderation and extract top genes:

```
fit_limma <- eBayes(fit_limma) # Empirical Bayes of standard errors

top_genes <- topTable(fit_limma, coef = "ER.statusPositive",
                      number = 10, sort.by = "P")

top_genes[, c("logFC", "AveExpr", "t", "P.Value", "adj.P.Val")]
```

##		logFC	AveExpr	t	P.Value	adj.P.Val
##	ENSG00000091831.22	3.944514	13.707728	37.07059	5.277717e-186	3.370139e-181
##	ENSG00000233823.1	4.847316	5.465671	28.71747	1.163808e-130	3.715806e-126
##	ENSG00000134830.5	2.208858	11.023674	28.67014	2.401626e-130	5.111942e-126
##	ENSG00000173467.8	5.402733	11.493465	28.28560	8.608629e-128	1.374282e-123
##	ENSG00000074410.13	2.679280	13.943950	28.11642	1.142811e-126	1.459507e-122
##	ENSG00000248360.7	3.266468	10.269532	27.97892	9.338883e-126	9.939062e-122
##	ENSG00000120262.9	2.301978	11.191298	27.56347	5.301287e-123	4.835986e-119

topTable ranks genes by statistical significance with:

- logFC: log fold change
- P.Value: p-value
- adj.P.Val: adjusted p-value (FDR correction)

What is eBayes?

Empirical Bayes (EB) moderation of variance estimates

The problem: With small sample sizes, gene-specific variance estimates $\hat{\sigma}_g^2$ are unreliable

- Some genes appear significant just by chance (unstable variance estimates)
- Leads to false discoveries

The solution: Borrow information across genes

- Assume all genes share similar variance structure
- Shrink each gene's variance toward a common prior distribution
- Results in more stable, moderated variance estimates

Empirical Bayes on what?

eBayes moderates the **variance** (or standard error) estimates:

$$\tilde{\sigma}_g^2 = \text{shrink}(\hat{\sigma}_g^2 \text{ toward prior})$$

Without eBayes: Each gene uses only its own data

- $\hat{\sigma}_g^2$ based on n samples for gene g alone
- Unstable when n is small

With eBayes: Each gene borrows strength from all genes

- Pools information across G genes to estimate prior distribution
- Shrinks extreme variance estimates toward the middle
- More stable t-statistics and p-values

Why is eBayes useful for many genes?

Traditional approach (gene by gene):

- Fit model separately for each gene
- Each has independent $\hat{\sigma}_g^2$
- High variance in estimates
- Many false positives/negatives

Empirical Bayes (sharing information):

- Fit models for all genes
- Moderate all $\hat{\sigma}_g^2$ together
- Stable variance estimates
- Better statistical power
- Reduced false discoveries

Key insight: With thousands of genes, we can learn a prior distribution from the data itself, then use it to improve individual gene estimates

Why use limma instead of regular lm?

- **Borrows information** from all genes to get a better estimate of the variance (especially in smaller sample size settings)
- Efficiently fits many regression models **without replicating unnecessary calculations!**
- Arranges output in a convenient way to ease further analysis, visualization, and interpretation

Genome-wide OLS fits

- Gene by gene:
 - `lm(y ~ x, data = gene)` for each gene
- All genes at once, using `limma`:
 - `lmFit(object, design)`
 - `object` matrix-like object with expression values for all genes
 - `design` is a specially formatted design matrix (more on this later)
 - Note that `object` can be a Bioconductor container, such as an `ExpressionSet` object

Today's lecture

- 1 What is a linear regression model?
- 2 Statistical inference for linear regression**
- 3 Generalized linear models for RNA-seq
- 4 The limma-voom approach

Recall: Linear model in matrix form

$$\mathbf{y}_g = \mathbf{X}\beta_g + \epsilon_g$$

where:

- $\mathbf{y}_g \in \mathbb{R}^n$: expressions for gene g
- $\mathbf{X} \in \mathbb{R}^{n \times p}$: design matrix (the same for all the genes)
- $\beta_g \in \mathbb{R}^p$: coefficients for gene g
- $\epsilon_g \in \mathbb{R}^n$: error terms

Probabilistic model for each sample

For each sample i and gene g :

$$Y_{ig} = \mathbf{x}_i \beta_g + \epsilon_{ig}$$

where $\mathbf{x}_i$ is the i -th row of $\mathbf{X}$

Assumption: $\epsilon_{ig} \sim \mathcal{N}(0, \sigma_g^2)$ independently

Probabilistic model for each sample

For each sample i and gene g :

$$Y_{ig} = \mathbf{x}_i \beta_g + \epsilon_{ig}$$

where $\mathbf{x}_i$ is the i -th row of $\mathbf{X}$

Assumption: $\epsilon_{ig} \sim \mathcal{N}(0, \sigma_g^2)$ independently

$$Y_{ig} \sim \mathcal{N}(\mathbf{x}_i \beta_g, \sigma_g^2)$$

The probability density for each sample

$$p(Y_{ig} \mid \beta_g, \sigma_g^2) = \frac{1}{\sqrt{2\pi\sigma_g^2}} \exp \left(-\frac{(Y_{ig} - \mathbf{x}_i\beta_g)^2}{2\sigma_g^2} \right)$$

Likelihood function

For all n samples (assuming independence):

$$L(\beta_g, \sigma_g^2 \mid \mathbf{y}_g) = \prod_{i=1}^n p(Y_{ig} \mid \beta_g, \sigma_g^2)$$

Likelihood function

For all n samples (assuming independence):

$$L(\beta_g, \sigma_g^2 \mid \mathbf{y}_g) = \prod_{i=1}^n p(Y_{ig} \mid \beta_g, \sigma_g^2)$$

Log-likelihood:

$$\begin{aligned}\ell(\beta_g, \sigma_g^2) &= \sum_{i=1}^n \log p(Y_{ig} \mid \beta_g, \sigma_g^2) \\ &= -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log(\sigma_g^2) - \frac{1}{2\sigma_g^2} \sum_{i=1}^n (Y_{ig} - \mathbf{x}_i \beta_g)^2\end{aligned}$$

Maximum likelihood estimation

To find $\hat{\beta}_g$, maximize the log-likelihood:

$$\hat{\beta}_g = \arg \max_{\beta_g} \ell(\beta_g, \sigma_g^2)$$

Maximum likelihood estimation

To find $\hat{\beta}_g$, maximize the log-likelihood:

$$\hat{\beta}_g = \arg \max_{\beta_g} \ell(\beta_g, \sigma_g^2)$$

Key observation: Maximizing ℓ is equivalent to minimizing:

$$\sum_{i=1}^n (Y_{ig} - \mathbf{x}_i \beta_g)^2 = (\mathbf{y}_g - \mathbf{X} \beta_g)^\top (\mathbf{y}_g - \mathbf{X} \beta_g)$$

Maximum likelihood estimation

To find $\hat{\beta}_g$, maximize the log-likelihood:

$$\hat{\beta}_g = \arg \max_{\beta_g} \ell(\beta_g, \sigma_g^2)$$

Key observation: Maximizing ℓ is equivalent to minimizing:

$$\sum_{i=1}^n (Y_{ig} - \mathbf{x}_i \beta_g)^2 = (\mathbf{y}_g - \mathbf{X} \beta_g)^\top (\mathbf{y}_g - \mathbf{X} \beta_g)$$

Maximum Likelihood = Ordinary Least Squares!

Under Gaussian errors, MLE and OLS give the same coefficient estimates

Ordinary Least Squares solution

Minimize $(\mathbf{y}_g - \mathbf{X}\beta_g)^\top (\mathbf{y}_g - \mathbf{X}\beta_g)$ with respect to β_g

Take derivative and set to zero:

$$\begin{aligned}\frac{\partial}{\partial \beta_g} [(\mathbf{y}_g - \mathbf{X}\beta_g)^\top (\mathbf{y}_g - \mathbf{X}\beta_g)] &= 0 \\ -2\mathbf{X}^\top \mathbf{y}_g + 2\mathbf{X}^\top \mathbf{X}\beta_g &= 0\end{aligned}$$

OLS estimator:

$$\hat{\beta}_g = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}_g$$

Beyond point estimates: Uncertainty quantification

So far we have $\hat{\beta}_g$ as a **point estimate**

But we also need:

- Standard errors for $\hat{\beta}_g$
- Confidence intervals
- Hypothesis tests (p-values)

Beyond point estimates: Uncertainty quantification

So far we have $\hat{\beta}_g$ as a **point estimate**

But we also need:

- Standard errors for $\hat{\beta}_g$
- Confidence intervals
- Hypothesis tests (p-values)

Key question: What is the distribution of $\hat{\beta}_g$?

Since $\hat{\beta}_g$ depends on $\mathbf{y}_g$, and $\mathbf{y}_g$ is random, $\hat{\beta}_g$ is also a random variable!

Variance of the coefficient estimates

$$\begin{aligned}\hat{\beta}_g &= (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{X} \beta_g + \epsilon_g) \\ &= \beta_g + (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \epsilon_g\end{aligned}$$

Variance of the coefficient estimates

$$\begin{aligned}\hat{\beta}_g &= (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{X} \beta_g + \epsilon_g) \\ &= \beta_g + (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \epsilon_g\end{aligned}$$

Variance:

$$\text{Var}(\hat{\beta}_g) = \text{Var}[(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \epsilon_g]$$

Since $\text{Var}(\epsilon_g) = \sigma_g^2 \mathbf{I}$:

$$\boxed{\text{Var}(\hat{\beta}_g) = \sigma_g^2 (\mathbf{X}^\top \mathbf{X})^{-1}}$$

Inference for coefficients

For the j -th coefficient of gene g :

$$\hat{\beta}_{jg} \sim \mathcal{N}(\beta_{jg}, \sigma_g^2 [(\mathbf{X}^\top \mathbf{X})^{-1}]_{jj})$$

Standard error: $\text{SE}(\hat{\beta}_{jg}) = \hat{\sigma}_g \sqrt{[(\mathbf{X}^\top \mathbf{X})^{-1}]_{jj}}$

t-statistic for $H_0 : \beta_{jg} = 0$:

$$t_{jg} = \frac{\hat{\beta}_{jg}}{\text{SE}(\hat{\beta}_{jg})} \sim t_{n-p}$$

Knowing the distribution each coefficient

t-statistic for $H_0 : \beta_{jg} = 0$:

$$t_{jg} = \frac{\hat{\beta}_{jg}}{\text{SE}(\hat{\beta}_{jg})} \sim t_{n-p}$$

This allows us to:

- Test significance of individual coefficients
- Construct confidence intervals

Today's lecture

- 1 What is a linear regression model?
- 2 Statistical inference for linear regression
- 3 Generalized linear models for RNA-seq**
- 4 The limma-voom approach

Limitation of Gaussian linear models for count data

Problem: RNA-seq data are **counts**, not continuous values

Gaussian assumption:

- $Y_{ig} \sim \mathcal{N}(\mu_{ig}, \sigma_g^2)$
- Continuous values
- Can be negative
- Constant variance

RNA-seq reality:

- $Y_{ig} \in \{0, 1, 2, 3, \dots\}$
- Discrete counts
- Non-negative integers
- Variance increases with mean (overdispersion)

Solution: Generalized Linear Models (GLM) with appropriate distribution

Negative Binomial for RNA-seq (Anders and Huber 2010; Robinson and Smyth 2008)

$$Y_{ig} \sim \text{NB}(\mu_{ig}, \phi_g)$$

Where:

- μ_{ig} : mean expression for gene g in sample i
- ϕ_g : dispersion parameter for gene g (captures overdispersion)

Key property:

$$\mathbb{E}[Y_{ig}] = \mu_{ig}, \quad \text{Var}(Y_{ig}) = \mu_{ig} + \phi_g \mu_{ig}^2$$

Connecting covariates to the mean: log-link function

Linear model (what we've been doing):

$$\mu_{ig} = \mathbf{x}_i \beta_g = \sum_j X_{ij} \beta_{jg}$$

Connecting covariates to the mean: log-link function

Linear model (what we've been doing):

$$\mu_{ig} = \mathbf{x}_i \beta_g = \sum_j X_{ij} \beta_{jg}$$

GLM solution: Use a **link function** to ensure $\mu_{ig} > 0$

$$\log(\mu_{ig}) = \mathbf{x}_i \beta_g = \sum_j X_{ij} \beta_{jg}$$

Connecting covariates to the mean: log-link function

Linear model (what we've been doing):

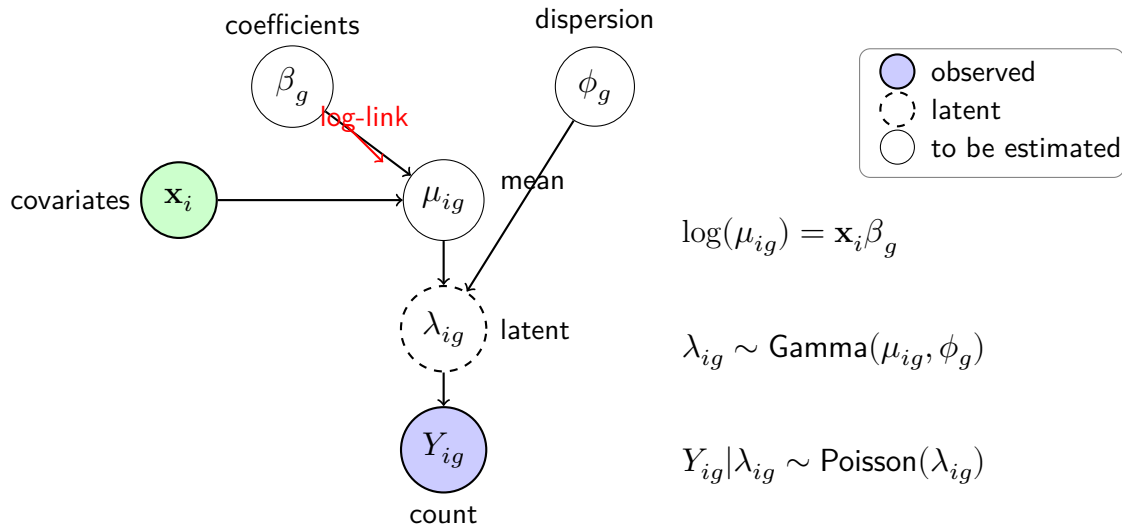
$$\mu_{ig} = \mathbf{x}_i \beta_g = \sum_j X_{ij} \beta_{jg}$$

GLM solution: Use a **link function** to ensure $\mu_{ig} > 0$

$$\log(\mu_{ig}) = \mathbf{x}_i \beta_g = \sum_j X_{ij} \beta_{jg}$$

$$\mu_{ig} = \exp \left(\sum_j X_{ij} \beta_{jg} \right) = \exp(\mathbf{x}_i \beta_g)$$

GLM for RNA-seq: putting it together



Interpretation of coefficients in GLM

In the log-link GLM:

$$\log(\mu_{ig}) = \beta_{0g} + \beta_{1g}X_{i1} + \beta_{2g}X_{i2} + \dots$$

Interpretation of β_{jg} :

- β_{jg} : **log fold-change** in mean expression for a one-unit Δ in X_{ij}

Interpretation of coefficients in GLM

In the log-link GLM:

$$\log(\mu_{ig}) = \beta_{0g} + \beta_{1g}X_{i1} + \beta_{2g}X_{i2} + \dots$$

Interpretation of β_{jg} :

- β_{jg} : **log fold-change** in mean expression for a one-unit Δ in X_{ij}

Example: If $\beta_{1g} = 0.693$ (for a binary treatment variable):

$$\mu_{ig}^{\text{treated}} = \mu_{ig}^{\text{control}} \times \exp(0.693) = \mu_{ig}^{\text{control}} \times 2$$

Treatment doubles the mean expression (2-fold change)

Comparing Gaussian LM vs. GLM for RNA-seq

	Gaussian LM	NB GLM
Distribution	$Y_{ig} \sim \mathcal{N}(\mu_{ig}, \sigma_g^2)$	$Y_{ig} \sim \text{NB}(\mu_{ig}, \phi_g)$
Mean model	$\mu_{ig} = \mathbf{x}_i \beta_g$	$\log(\mu_{ig}) = \mathbf{x}_i \beta_g$
Link function	Identity	Log
β_{jg} interpretation	Additive effect	Log fold-change
Variance	Constant σ_g^2	Increases with mean
Data type	Continuous	Count

For RNA-seq: GLM with negative binomial distribution is more appropriate!

DESeq2 (Love et al. 2014) / edgeR (Robinson and Smyth 2008) directly model count data

- ① Use negative binomial GLM
- ② Estimate dispersion parameters
- ③ Perform inference on β_g

Example: edgeR for negative binomial GLM

```
.file <- file.path(data.dir, "edgeR_results.rds")
if.needed(.file, {
  # Create DGEList object with count data
  dge <- DGEList(counts = raw.counts[, complete_samples],
                 samples = clinical.data[complete_samples, ])

  # Normalize for library size differences
  dge <- calcNormFactors(dge)

  # Estimate dispersion (phi_g for each gene)
  dge <- estimateDisp(dge, design)

  # Fit negative binomial GLM
  QLfit <- glmQLFit(dge, design = design)

  # Test for ER status effect
  QLtest_ER <- glmQLFTest(QLfit, coef = "ER.statusPositive")

  saveRDS(QLtest_ER, .file)
})
QLtest_ER <- readRDS(.file)

topTags(QLtest_ER)[1:3, c("logFC", "logCPM", "PValue", "FDR")]
```

```
## Coefficient: ER.statusPositive
##           logFC      logCPM      PValue      FDR
## ENSG00000228295.1 -7.201245 -3.01982535 8.275993e-261 5.284718e-256
## ENSG00000109208.4 -8.669208 -0.03401804 3.133466e-206 1.000453e-201
## ENSG00000236892.1 -6.484945 -4.80710360 1.004212e-205 2.137500e-201
```

Example: DESeq2 for negative binomial GLM

```
.file <- file.path(data.dir, "deseq2_results.rds")
if.needed(.file, {
  # Create DESeqDataSet from count matrix
  dds <- DESeqDataSetFromMatrix(
    countData = raw.counts[, complete_samples],
    colData = clinical.data[complete_samples, ],
    design = ~ age + menopause + ER.status
  )

  # Run DESeq2 pipeline (normalization + dispersion + GLM)
  dds <- DESeq(dds, quiet = TRUE)

  # Extract results for ER status
  res_ER <- results(dds, name = "ER.status_Positive_vs_Negative")

  saveRDS(res_ER, .file)
})
res_ER <- readRDS(.file)

res_ER[order(res_ER$padj), ][1:3, c("log2FoldChange", "pvalue", "padj")]
```

```
## log2 fold change (MLE): ER.status Positive vs Negative
## Wald test p-value: ER.status Positive vs Negative
## DataFrame with 3 rows and 3 columns
##           log2FoldChange      pvalue      padj
##           <numeric>      <numeric>      <numeric>
## ENSG00000091831.22      4.03638 4.09387e-239 1.61761e-234
## ENSG00000134830.5       2.80309 5.08558e-215 1.00473e-210
## ENSG00000171428.13      4.20200 1.79771e-199 2.36776e-195
```

How DESeq2 estimates dispersion parameter ϕ

The challenge: Estimating ϕ_g from small samples is unreliable

DESeq2's solution: Empirical Bayes shrinkage

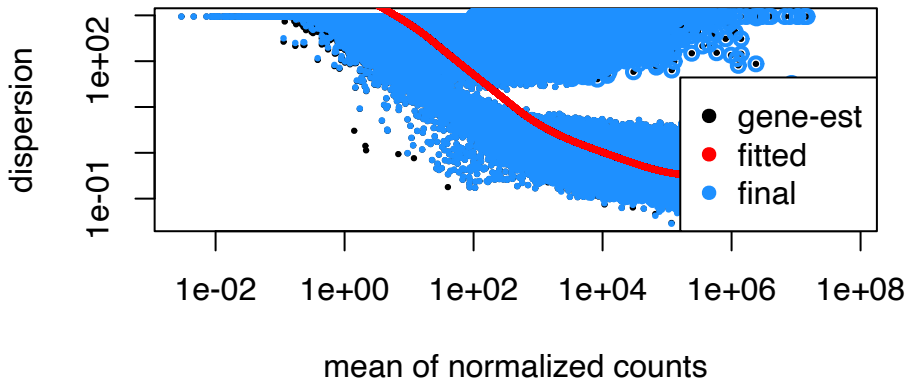
- 1 Fit gene-wise dispersions (MLE)
- 2 Fit a trend: $\phi_g \sim f(\mu_g)$
- 3 Shrink toward the trend

DESeq2 dispersion estimates

```
.file <- file.path(data.dir, "dds_full.rds")
if.needed(.file, {
  dds <- DESeqDataSetFromMatrix(
    countData = raw.counts[, complete_samples],
    colData = clinical.data[complete_samples, ],
    design = ~ age + menopause + ER.status
  )
  dds <- DESeq(dds, quiet = TRUE)
  saveRDS(dds, .file)
})
dds <- readRDS(.file)

plotDispEsts(dds, main = "DESeq2 Dispersion Estimates")
```

DESeq2 Dispersion Estimates



Today's lecture

- 1 What is a linear regression model?
- 2 Statistical inference for linear regression
- 3 Generalized linear models for RNA-seq
- 4 The limma-voom approach**

Two main approaches for RNA-seq differential expression



Count-based GLM

- Model counts directly with NB
- Estimate dispersion ϕ_g
- Use GLM framework



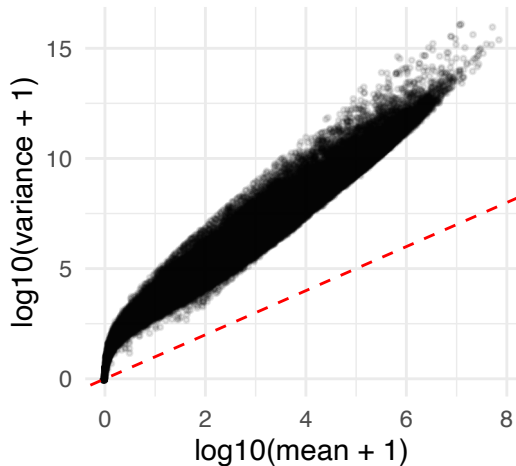
Transformation + Linear Model

- Transform counts to log-scale
- Apply weighted linear models
- Use **voom** for weights

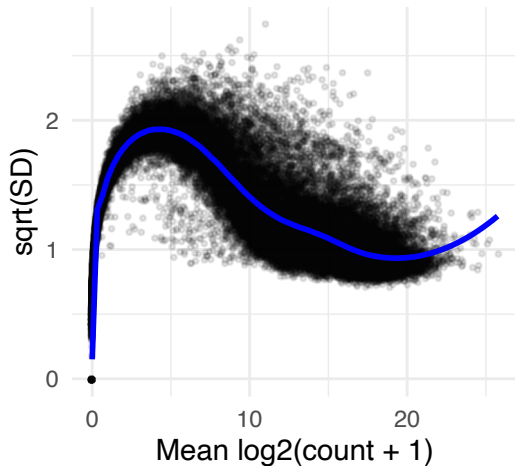
Both approaches give similar results in practice! But DESeq2 is ...

Mean-variance relationship and heteroskedasticity

Raw: Strong M-V trend



Log counts



Why does heteroskedasticity matter?

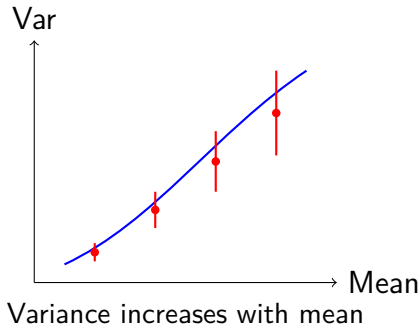
OLS assumes constant variance (homoskedasticity)

When variance depends on mean:

- High-variance observations get **less weight** than they should
- Standard errors are **incorrect**
- t-statistics and **p-values are wrong**

For RNA-seq:

- Low-count genes have high variance
- They get inflated t-statistics
- **More false positives!**



The voom solution: Precision weights

Key idea: Model the mean-variance relationship, use it to weight observations

$$w_{ig} = \frac{1}{\text{Var}(Y_{ig})}$$

- Observations with **high variance** get **low weight**
- Observations with **low variance** get **high weight**
- Weights are estimated from the **mean-variance trend**

This is called **Weighted Least Squares (WLS)**

Weighted Least Squares

OLS solution:

$$\hat{\beta}_g = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}_g$$

WLS solution:

$$\hat{\beta}_g = (\mathbf{X}^\top \mathbf{W}_g \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W}_g \mathbf{y}_g$$

where $\mathbf{W}_g = \text{diag}(w_{1g}, w_{2g}, \dots, w_{ng})$

Intuition: In minimizing RSS, put less weight on imprecise observations

The voom algorithm: Step by step

Step 1: Fit linear model to $\log_2(\text{CPM} + 0.5)$

Step 2: Extract residual standard deviations s_g

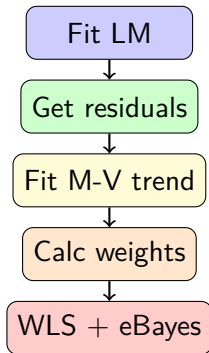
Step 3: Fit lowess curve to mean vs $\sqrt{s_g}$ trend

Step 4: Compute precision weights:

$$w_{ig} = \frac{1}{\hat{f}(\hat{\mu}_{ig})^4}$$

Step 5: Re-fit with weights using `lmFit`

Step 6: Apply eBayes for moderated t-statistics

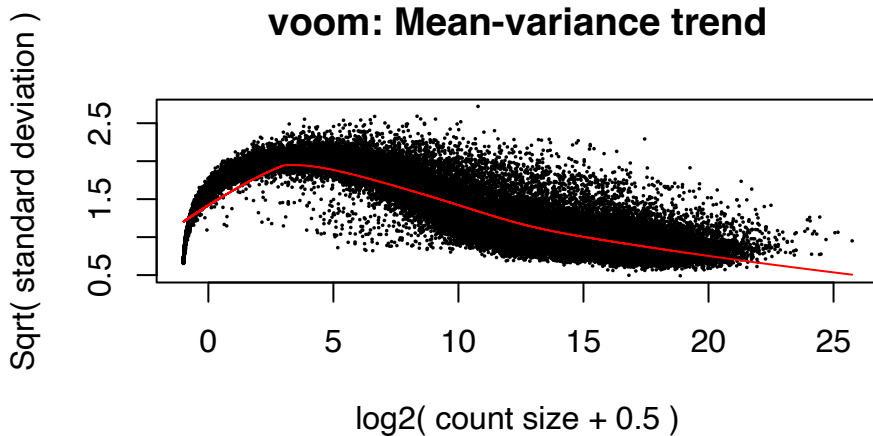


voom in action: TCGA BRCA data

```
## just like edgeR
dge <- DGEList(counts = raw.counts[, complete_samples])
dge <- calcNormFactors(dge)

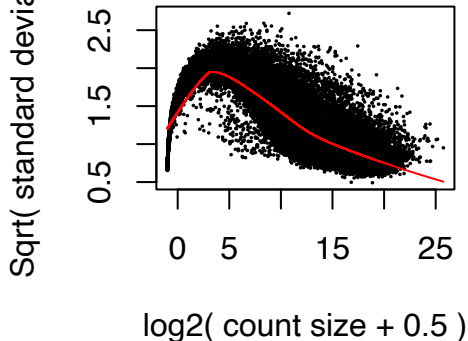
## VOOM transformation
voom_result <- voom(dge, design = design, plot = TRUE)
```

voom in action: TCGA BRCA data



Understanding the voom plot

voom: Mean-variance trend



X-axis: Average log-expression (mean of $\log_2$ CPM)

Y-axis: $\sqrt{SD}$ of residuals (quarter-root variance)

Red curve: Lowess fit to the trend

Interpretation:

- Downward trend = heteroskedasticity
- Low-expression genes have high variance
- Weights correct for this

Fitting the weighted linear model

```
# Fit linear model with voom weights
```

```
fit_voom <- lmFit(voom_result, design)
```

```
# Apply empirical Bayes moderation
```

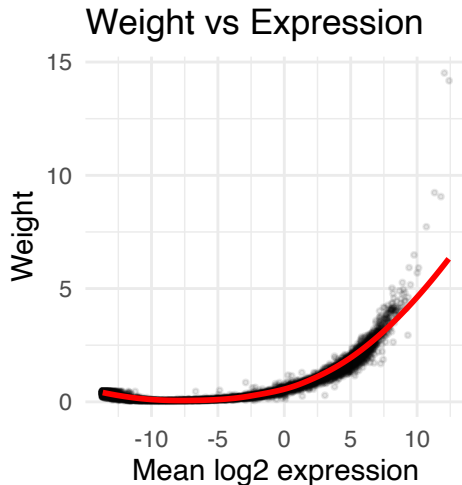
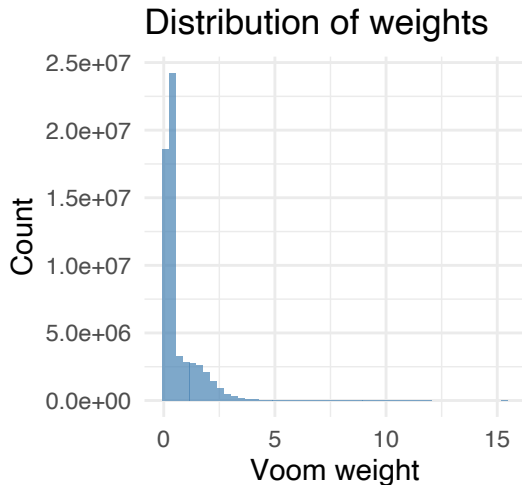
```
fit_voom <- eBayes(fit_voom)
```

```
# Get top genes for ER status
```

```
topTable(fit_voom, coef = "ER.statusPositive", number = 5)[,  
  c("logFC", "AveExpr", "t", "P.Value", "adj.P.Val")]
```

##		logFC	AveExpr	t	P.Value	adj.P.Val
##	ENSG00000135069.13	-3.462699	1.903421	-28.85150	1.253745e-131	8.005916e-127
##	ENSG00000207422.1	7.296185	-6.671348	26.59420	1.176558e-116	3.756516e-112
##	ENSG00000091831.22	5.693097	7.103247	25.81213	1.669790e-111	3.554203e-107
##	ENSG00000053524.12	-2.095270	1.667369	-25.76217	3.556226e-111	5.677159e-107
##	ENSG00000176597.11	-2.186182	2.937231	-25.55288	8.414932e-110	1.074688e-105

What are the voom weights?



Higher expression $\rightarrow$ higher weight (more precise)

limma-voom vs limma-trend

limma-voom

- Weights at **observation** level
- Each sample-gene gets its own weight
- More flexible

```
voom_result <- voom(dge, design)
fit <- lmFit(voom_result, design)
fit <- eBayes(fit)
```

limma-trend

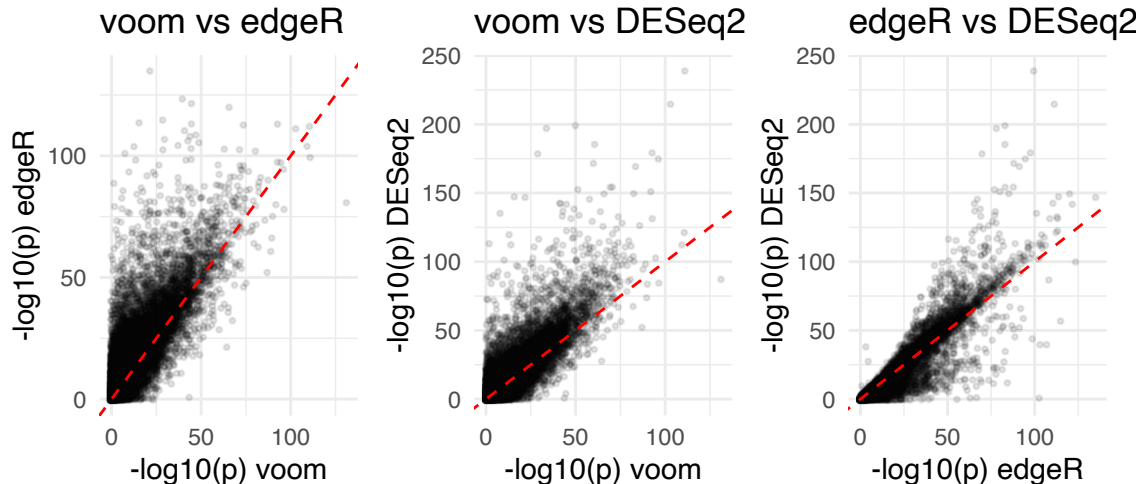
- Shrinkage at **gene** level
- Prior variance depends on mean expression
- Simpler, often sufficient

```
fit <- lmFit(logCPM, design)
fit <- eBayes(fit, trend = TRUE)
```

When to use which?

- voom: When library sizes vary a lot
- trend: When library sizes are similar

Comparing all three methods on TCGA BRCA



Methods agree well! Especially for top-ranked genes

Summary: When to use which method?

	limma-voom	DESeq2	edgeR
Data type	Counts	Counts	Counts
Model	Weighted LM	NB GLM	NB GLM
M-V handling	voom weights	Dispersion	Dispersion
Speed	Fast	Moderate	Fast
Small samples	Good	Very good	Very good
Large samples	Very good	Good	Good

Practical advice:

- All three methods are valid and well-tested
- For most datasets, results will be similar
- Use DESeq2/edgeR for small samples ($n < 10$ per group)
- Use limma-voom for large samples or when library sizes vary

Key takeaways

- ① **RNA-seq counts** have mean-variance relationship (heteroskedasticity)
- ② **voom** estimates precision weights from M-V trend
- ③ **Weighted least squares** gives more accurate inference
- ④ **DESeq2/edgeR** model counts directly with negative binomial
- ⑤ **All methods** borrow information across genes (empirical Bayes)
- ⑥ **Results are similar** for well-powered studies

Actual key takeaway

Analysis Assignment

Reference

- Anders, Simon, and Wolfgang Huber. 2010. "Differential Expression Analysis for Sequence Count Data." *Genome Biol.* 11 (10): R106.
- Love, Michael I, Wolfgang Huber, and Simon Anders. 2014. "Moderated Estimation of Fold Change and Dispersion for RNA-Seq Data with DESeq2." *Genome Biol.* 15 (12): 1–21.
- Robinson, Mark D, and Gordon K Smyth. 2008. "Small-Sample Estimation of Negative Binomial Dispersion, with Applications to SAGE Data." *Biostatistics* 9 (2): 321–32.